CQRS and Event Sourcing in Microservices

build the view from events

Watch the 4-min video

> **In one line:** to query across services you either compose their APIs at read time, or build a dedicated **read model** from their events: a projection shaped for the screen, fast and resilient but eventually consistent, and rebuildable by replaying the event history.

1 One screen, four databases

Support wants one screen with a customer's profile, orders, payments and tickets. That data lives in four services, each with its own database (lesson 23), so there is no join. How do you answer a query that spans services?

2 CQRS across services

CQRS (command query responsibility segregation) splits the **write side**, which handles commands and enforces rules, from the **read side**, which serves queries from models shaped for the screen. In microservices the read model doesn't have to live in the owning service. Each service publishes events when its data changes, and a separate **projection** service consumes them and maintains one denormalised view per customer. Queries hit that view, with no joins at query time.

Event sourcing goes further inside a service: the events themselves are the source of truth (an append-only log), and current state is derived by replaying them. You don't need event sourcing to build cross-service read models; published integration events are enough, as long as they are kept long enough to replay.

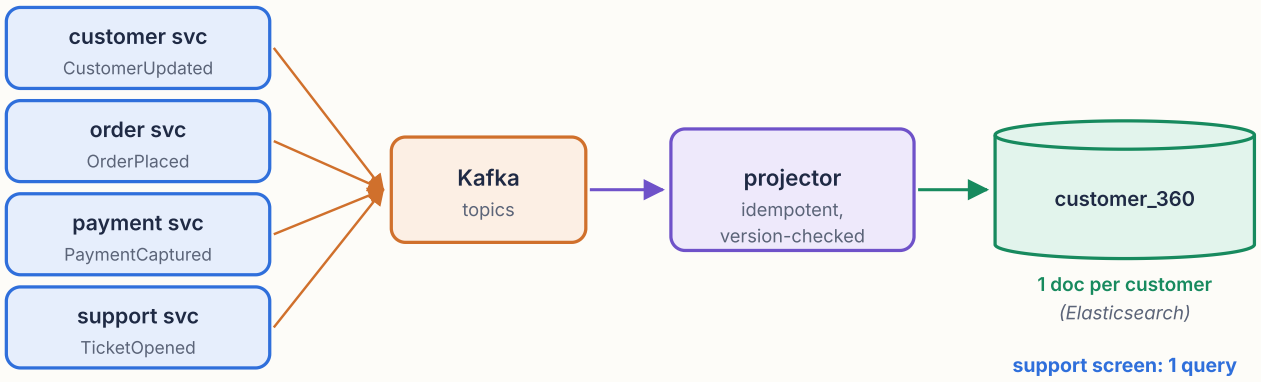


Figure 1. Services publish events; a projector folds them into one document per customer; the screen reads one document.

3 Composition or read model?

	API composition	Event-built read model
How	call all four services at query time and merge	projector maintains a view from events
Freshness	always current	eventually consistent (lag: ms to seconds)
Latency	slowest service + merge	one lookup
One service down	query fails or is partial	still served (slightly stale)
Cost	simple	broker, projector, extra store, rebuilds

Many teams start with composition and add read models for hot or complex queries.

4 Projections done right

- **Idempotent:** events can arrive twice; record processed event IDs or use upserts.
- **Ordering:** handle out-of-order events with per-entity version numbers; ignore an event older than the stored version.
- **Rebuilds:** to fix a bug or add a field, replay the history into a **new** table or index while the old one keeps serving, then switch. This needs events retained long enough (e.g. Kafka retention or compacted topics).
- **Operations:** monitor **projection lag** and show staleness if needed; version event schemas; remember the view is a copy and the owning services stay the source of truth.

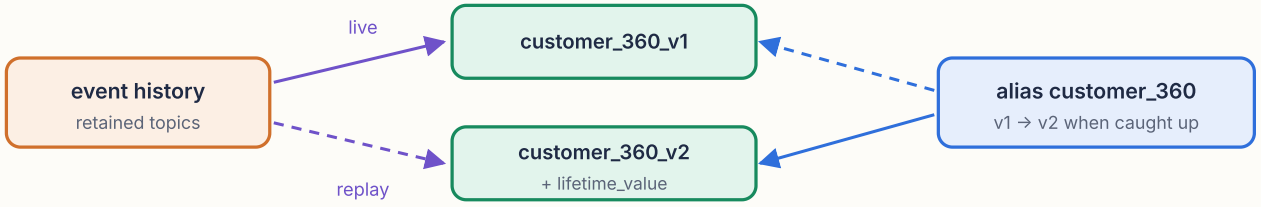


Figure 2. Safe change: build v2 from history beside v1, then move the alias. Users never see a half-built view.

WORKED EXAMPLE: THE CUSTOMER 360 VIEW

Customer, order, payment and support services publish events to Kafka. A projector writes one document per customer into Elasticsearch, so the support screen loads in a single call. To add a `lifetime_value` field, the team deploys a new projector that replays the retained topics into `customer_360_v2`. When its lag reaches zero they switch the `customer_360` alias from v1 to v2, and delete v1 a week later.

COMMON MISTAKES

- Treating the read model as the source of truth and writing to it.
- Non-idempotent projectors that double-count on redelivery.
- Event retention too short to rebuild a projection.
- Building read models for queries that simple composition would serve.

5 Key terms

CQRS: separate models for writes (commands) and reads (queries)

Event sourcing: events are the source of truth; state is derived

Projection lag: delay between an event and the updated view

Read model / projection: query-shaped view built from events

API composition: query several services and merge at read time

Replay: rebuild a view by reprocessing event history

6 Interview questions

Q1. What is CQRS? BASIC

Separating the write model, which handles commands and enforces rules, from read models shaped for queries, so each can be designed and scaled for its job.

Q2. Is event sourcing required for CQRS? BASIC

No: read models can be built from published integration events; event sourcing makes the event log the write side's source of truth.

Q3. How do you build a query that spans multiple services? SCENARIO

Either API composition at query time (simple, fresh, but slower and fragile) or a read model built from the services' events (fast and resilient, eventually consistent).

Q4. How do you change a read model safely? SCENARIO

Replay the event history into a new projection (new table or index) beside the old one, and switch traffic, e.g. an alias, once it has caught up.

Q5. An event arrives twice and another arrives out of order. How does the projector cope? TRICKY

It is idempotent (dedupe by event ID or upsert) and checks a per-entity version, ignoring events older than what it has applied.

Q6. Support sees an order the customer cancelled a second ago. Bug? TRICKY

Expected eventual consistency: the view lags the source; monitor lag, show a freshness hint, or read from the owning service for critical checks.

Next: SD-L-018 · Distributed Tracing in Microservices backendroadmap.com